Enterprise Database Engineering Assignment

Views, Procedures, Triggers & Scheduler Jobs

Part 1 – Research & Understanding:

1. Views:

1.1 What is a View?

A View is a virtual table defined by a stored SQL SELECT statement. It does not physically store data — it retrieves data from the underlying tables each time it is queried. From a user's perspective, a View looks and behaves like a regular table.

1.2 Why do companies use Views?

- To simplify complex queries by hiding JOIN logic behind a single name.
- To restrict user access to specific columns or rows without changing table permissions.
- To maintain consistent results across multiple applications using the same logic.
- To abstract the physical table structure from end users and applications.

1.3 What security benefits do Views provide?

- Hide sensitive columns such as salary or personal data from unauthorized users.
- Grant SELECT access to a View without exposing the underlying tables.
- Restrict rows shown to each user based on department or role.
- Prevent users from directly modifying base table data.

1.4 Differences: Table vs View vs Materialized View :

Feature	Table	View	Materialized View
Stores data?	Yes – physically on disk	No – virtual only	Yes – physically on disk
Data freshness	Always current	Always current (live query)	Requires manual REFRESH
Performance	Fast	Depends on base query	Very fast (pre-computed)
Best used for	Storing raw data	Access control & simplicity	Large reporting queries

1.5 Enterprise Use Cases :

- Use Case 1 – HR Access Control: The HR department queries a View that shows only employee names and department names. Salary columns are excluded, protecting sensitive compensation data.
- Use Case 2 – Management Dashboard: A View joins the EMPLOYEES, DEPARTMENTS, and LOCATIONS tables and presents a consolidated summary for management without exposing the schema structure.
- Use Case 3 – Finance Reporting: The Finance team queries a View that returns aggregated salary data per department. No knowledge of the underlying table design is required.

- **2. Stored Procedures:**

2.1 What is a Stored Procedure?

A Stored Procedure is a named, pre-compiled block of PL/SQL code saved inside the database. It accepts input parameters, executes SQL logic, and can return output through OUT parameters. It is called by name and runs on the database server.

2.2 Why do companies use Stored Procedures?

- To reuse the same logic across multiple applications without rewriting it.
- To improve performance through pre-compiled and cached execution plans.
- To enforce security by letting users execute operations without direct table access.
- To centralize business logic in one place for easier maintenance.

2.3 What problems do Procedures solve?

- Code duplication: The same query is written once and called everywhere.
- Inconsistency: All applications use the same business logic with the same results.
- Maintenance: When a rule changes, only the Procedure needs updating, not every application.
- Security gaps: Users cannot execute arbitrary SQL — they can only call defined Procedures.

2.4 Differences: Procedure vs Function :

Feature	Procedure	Function
Returns a value?	Not required (uses OUT params)	Always returns one value
Used in SQL SELECT?	No	Yes – can be called inline
Primary purpose	Perform operations / actions	Compute and return a result
Example	Process payroll for a department	Calculate annual salary from monthly

2.5 Enterprise Use Cases:

- Use Case 1 – Payroll Processing: A Procedure accepts a department ID and calculates the monthly salary total for all employees in that department, called by Finance at month-end.
- Use Case 2 – Employee Onboarding: A Procedure inserts a new employee record into multiple tables (EMPLOYEES, JOB_HISTORY) in a single atomic operation, ensuring data consistency.
- Use Case 3 – Bonus Calculation: A reusable Procedure calculates annual bonuses based on salary and performance. Any department or system calls this one Procedure to get consistent results.

3. Triggers:

3.1 What is a Trigger?

A Trigger is a PL/SQL block that fires automatically when a specific DML event (INSERT, UPDATE, or DELETE) occurs on a table. Triggers are never called manually — they execute silently in the background whenever the defined event happens.

3.2 Why do companies use Triggers?

- To automatically record audit trails whenever sensitive data changes.
- To enforce data integrity rules at the database level, independent of the application.
- To automate follow-up actions such as updating a related table after a change.
- To ensure business rules apply regardless of which application made the change.

3.3 Differences: BEFORE Trigger vs AFTER Trigger:

Feature	BEFORE Trigger	AFTER Trigger
When it fires	Before the DML executes	After the DML completes
Can modify :NEW values?	Yes – data can be changed before saving	No – data is already saved
Primary use	Validation and data transformation	Auditing and logging
Example	Block a salary below minimum wage	Record salary change in audit table

3.4 Risks of excessive Trigger usage:

- Performance: Every DML operation fires the Trigger, adding processing overhead to busy tables.
- Difficult debugging: Triggers run invisibly. Unexpected data changes are hard to trace.
- Cascading Triggers: A Trigger may cause another DML that fires another Trigger, creating hard-to-control chains.
- Hidden logic: Business rules buried in Triggers are easy to miss, causing confusion during system changes.

3.5 Enterprise Use Cases:

- Use Case 1 – Salary Audit: An AFTER UPDATE Trigger on EMPLOYEES automatically inserts a record into SALARY_AUDIT with the old salary, new salary, date, and username whenever a salary is changed.
- Use Case 2 – Data Validation: A BEFORE INSERT Trigger checks that a new employee's salary is not below the legal minimum. If it is, the Trigger raises an error and blocks the insert.
- Use Case 3 – Deletion Logging: A BEFORE DELETE Trigger copies the deleted row into an archive table before the deletion occurs, ensuring the record is never permanently lost.

4. Scheduler Jobs:

4.1 What is a Scheduler Job?

An Oracle Scheduler Job is an automated task managed by DBMS_SCHEDULER that runs at a defined time or on a recurring schedule without any manual intervention. It is used to automate repetitive database operations such as generating reports, cleaning data, or refreshing Materialized Views.

4.2 Why do companies use Scheduler Jobs?

- To automate repetitive tasks and eliminate manual execution.
- To ensure tasks run consistently and reliably on schedule.
- To schedule heavy processes during off-peak hours to avoid impacting business operations.
- To free IT teams from running routine maintenance scripts manually.

4.3 Differences: Trigger vs Scheduler Job:

Feature	Trigger	Scheduler Job
What starts it?	A database event (INSERT/UPDATE/DELETE)	A defined time or recurring schedule
Runs how often?	Every time the DML event occurs	At the scheduled interval (daily, weekly)
Best used for	Real-time response to data changes	Batch processing and scheduled reports
Example	Record salary change immediately	Generate salary report every Friday at 4 PM

4.4 Commonly automated processes

- Generating weekly and monthly payroll and financial reports.
- Refreshing Materialized Views to update pre-computed report data.
- Purging old audit records that have exceeded the data retention period.
- Performing nightly database backups and archiving historical records.

4.5 Enterprise Use Cases:

- Use Case 1 – Weekly Salary Report: A Job runs every Friday at 4:00 PM and calls a Procedure that generates a salary summary report per department, stored in a REPORTS table for management review.
- Use Case 2 – Monthly Data Cleanup: On the first day of each month, a Job deletes audit records older than six months, keeping the SALARY_AUDIT table at a manageable size automatically.
- Use Case 3 – Nightly Backup: A Job runs every night at 2:00 AM to export critical tables to a backup location. A log entry confirms success; if the Job fails, an alert is automatically raised.

- **Summary:**

Object	What It Is	When to Use	Key Benefit
View	Virtual table (saved query)	Restrict or simplify data access	Security & simplification
Procedure	Reusable named PL/SQL block	Repeatable operations & business logic	Reusability & performance
Trigger	Auto-executing code on DML events	Auditing, validation, automation	Real-time automatic response
Scheduler Job	Time-based automated task	Recurring batch processes & reports	Automation & reliability

Part 2 – Enterprise Decision Making:

Scenario 1:

The HR department should only see Employee Name and Department Name. They should not see salary information.

Object to Use: View

Reasoning: A View is the right choice here because the requirement is about restricting what data users can see. A View is created that selects only the employee name and department name columns from the EMPLOYEES table. HR users are given access to the View only, not to the base table. This keeps salary data completely hidden without modifying any table structure.

Scenario 2:

Every salary update must automatically be recorded for auditing purposes.

Object to Use: Trigger

Reasoning: A Trigger is the right choice because the recording must happen automatically every time a salary is updated, regardless of which application or user made the change. An AFTER UPDATE Trigger on the EMPLOYEES table fires on every salary change and inserts the old salary, new salary, date, and username into a SALARY_AUDIT table. No manual action is needed.

Scenario 3:

Management wants a report generated automatically every Friday at 4:00 PM.

Object to Use: Scheduler Job

Reasoning: A Scheduler Job is the right choice because the requirement is time-based, not event-based. A Job is set up using DBMS_SCHEDULER to run every Friday at 4:00 PM. When it fires, it executes the logic needed to generate and store the report. This runs automatically with no manual effort required.

Scenario 4:

The Finance department wants one reusable process that calculates annual bonuses.

Object to Use: Stored Procedure

Reasoning: A Stored Procedure is the right choice because the requirement is a reusable operation with defined business logic. The Procedure is written once, accepts the needed input, applies the bonus calculation rules, and returns the results. Any application or department can call this same Procedure and always get consistent results.

Scenario 5:

The company wants a notification whenever an employee's salary is modified.

Object to Use: Trigger

Reasoning: A Trigger is the right choice because the notification must happen automatically and instantly at the moment the salary is changed. An AFTER UPDATE Trigger on the EMPLOYEES table detects every salary modification and logs the event into a notification table. This happens immediately without any changes needed in the application code.

Scenario 6:

Management wants a dashboard displaying employee information without exposing underlying tables.

Object to Use: View

Reasoning: A View is the right choice because the requirement is to present data in a controlled format while hiding the underlying table structure. A View joins the necessary tables (EMPLOYEES, DEPARTMENTS, JOBS) and exposes only the columns needed for the dashboard. Management queries the View directly with no access to or knowledge of the base tables.

Summary:

No:	Scenario	Object Used	Key Reason
1	HR sees name & department only (no salary)	View	Restrict column access
2	Every salary update must be recorded	Trigger	Auto-fires on UPDATE
3	Report every Friday at 4:00 PM	Scheduler Job	Time-based automation
4	Reusable bonus calculation process	Stored Procedure	Reusable business logic
5	Notification on every salary change	Trigger	Auto-fires on UPDATE
6	Dashboard without exposing base tables	View	Abstraction & access control

Part 3 – Architecture Design:

The design below describes a simple HR system architecture using Views, Stored Procedures, Triggers, and Scheduler Jobs. Each object has a specific role and interacts with the others to build a secure and automated system.

HR Users:

User / Role	Access Method
HR Staff	Queries VW_HR_EMPLOYEES View — sees name and department only, no salary.
Finance Team	Calls SP_CALC_BONUS Procedure and queries VW_DEPT_SALARY View.
Management	Queries VW_MANAGEMENT and VW_EMPLOYEE_DETAILS Views for the dashboard.
DBA	Has direct access to all tables, Triggers, Procedures, and Scheduler Jobs.

Database Objects:

Object Type	Name	Role
View	VW_HR_EMPLOYEES	Shows employee name and department only. Hides salary from HR staff.
View	VW_EMPLOYEE_DETAILS	Shows full employee info for management (name, job, department, salary).
View	VW_DEPT_SALARY	Shows average salary per department for Finance.
View	VW_MANAGEMENT	Joins EMPLOYEES, DEPARTMENTS, JOBS for the management dashboard.
Procedure	SP_GET_DEPT_EMPLOYEES	Returns all employees and salaries for a given department ID.
Procedure	SP_CALC_BONUS	Calculates annual bonus for employees in a given department.
Procedure	SP_SALARY_REPORT	Generates salary summary report. Called by Scheduler Jobs.
Trigger	TRG_SALARY_AUDIT	AFTER UPDATE — records every salary change into SALARY_AUDIT.
Trigger	TRG_DELETE_LOG	BEFORE DELETE — copies deleted employee row into CHANGE_LOG.
Scheduler Job	JOB_DAILY_SALARY	Runs every day at 8:00 PM. Calls SP_SALARY_REPORT and stores output.
Scheduler Job	JOB_WEEKLY_DEPT	Runs every Friday at 4:00 PM. Generates department salary ranking report.
Scheduler Job	JOB_MONTHLY_CLEANUP	Runs 1st of every month. Deletes SALARY_AUDIT records older than 6 months.

Base Tables (HR Schema) :

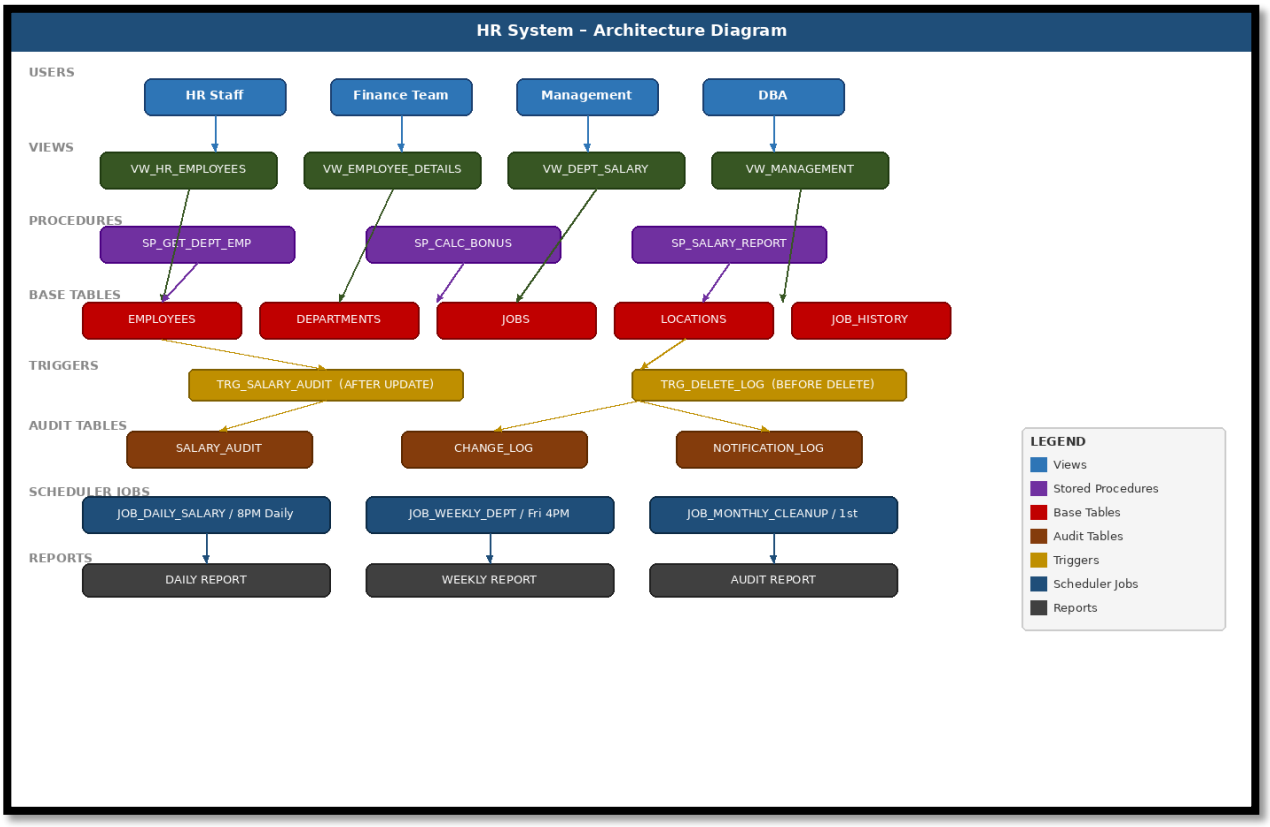
Table	Description
EMPLOYEES	Stores employee records including salary, job ID, and department ID.
DEPARTMENTS	Stores department names and manager IDs.
JOBS	Stores job titles and salary ranges.
LOCATIONS	Stores office locations linked to departments.
JOB_HISTORY	Stores historical records of employee job changes.

Audit Tables:

Table	Populated By	What It Stores
SALARY_AUDIT	TRG_SALARY_AUDIT Trigger	Employee ID, old salary, new salary, change date, username.
CHANGE_LOG	TRG_DELETE_LOG Trigger	Full row copy of any deleted employee record before deletion.
REPORTS	Scheduler Jobs	Generated report data stored for management review.

Architecture Diagram :

The diagram below shows how all components interact within the HR system.



How the Objects Interact :

- HR Staff query VW_HR_EMPLOYEES. The View returns only name and department from the EMPLOYEES table. Salary is never visible.
- Management query VW_MANAGEMENT and VW_EMPLOYEE_DETAILS. These Views join EMPLOYEES, DEPARTMENTS, and JOBS and return the required columns for the dashboard.
- Finance calls SP_CALC_BONUS or SP_GET_DEPT_EMPLOYEES. The Procedure reads from EMPLOYEES and returns the results without giving direct table access.
- When a salary is updated in EMPLOYEES, TRG_SALARY_AUDIT fires automatically and inserts a record into SALARY_AUDIT with the old value, new value, date, and username.
- When an employee is deleted, TRG_DELETE_LOG fires before the deletion and copies the full row into CHANGE_LOG for recovery.
- JOB_DAILY_SALARY runs every day at 8:00 PM, calls SP_SALARY_REPORT, and stores the output in the REPORTS table.
- JOB_WEEKLY_DEPT runs every Friday at 4:00 PM and generates the department salary ranking report.

JOB_MONTHLY_CLEANUP runs on the 1st of every month and removes SALARY_AUDIT records older than six months.

Part 4 – Reflection :

Question:
If applications can perform all business logic themselves, why do enterprise systems still place logic inside the database?

Enterprise systems place critical logic inside the database — not only in the application — for four fundamental reasons:

1. Consistency Across Multiple Applications :
Large organizations run many systems simultaneously (web apps, mobile apps, reporting tools, third-party integrations). If each application contains its own copy of the business logic, the rules will eventually diverge. A **Stored Procedure** written once in the database is called by every application and always produces the same result. There is no risk of one system calculating bonuses differently from another.

2. Security Cannot Be Enforced at the Application Layer Alone :
An application can be bypassed. A developer with database credentials can connect directly using SQL Developer or any tool and query any table. **Views** enforce column-level and row-level restrictions at the database level, meaning a user who connects through any tool will still only see the data the View allows. This protection cannot be disabled by changing application code.

3. Automatic Enforcement Regardless of Who Changes the Data:
Business rules placed inside an application only apply when that application is the one making the change. If a DBA updates a salary directly in SQL Developer, the application rule is completely skipped. A **Trigger** fires on every DML event regardless of the source — application, script, or direct connection. Audit logging through a Trigger is guaranteed to capture every change with no exceptions.

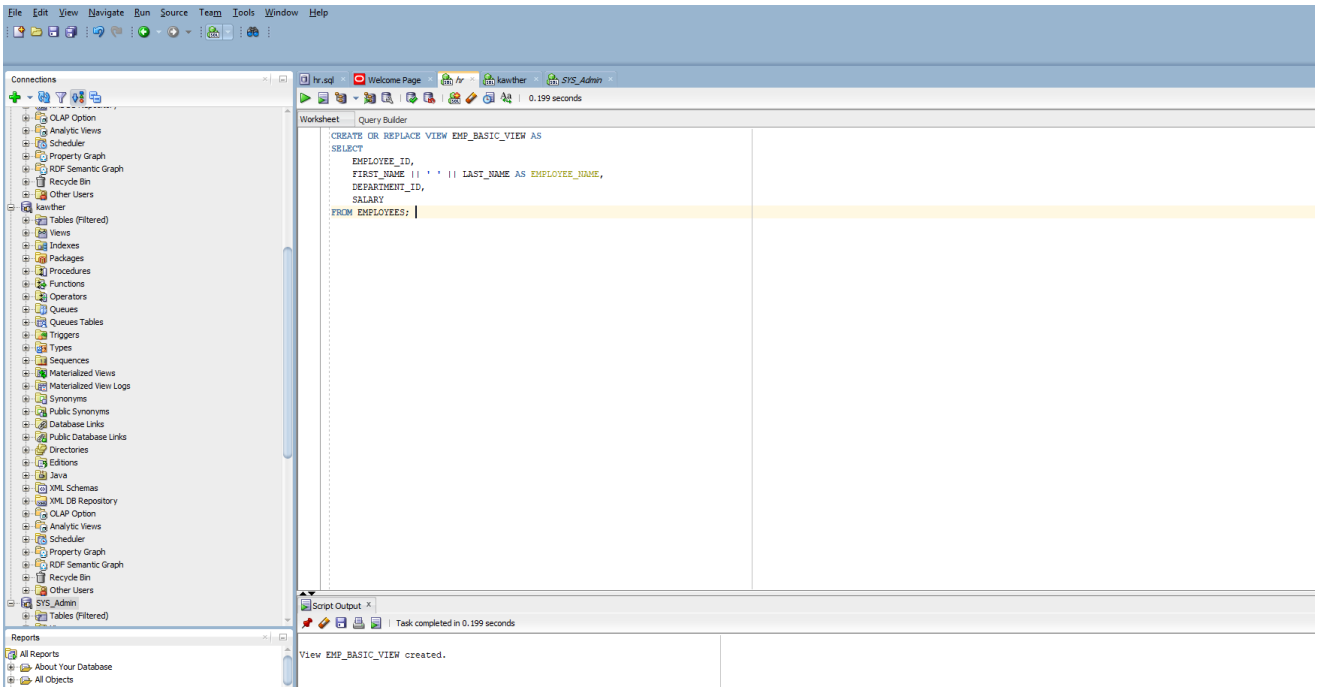
4. Automation Independent of Application Availability:
Scheduled tasks such as report generation, data cleanup, and payroll processing must run at defined times even when no users are logged in and no application server is running. A **Scheduler Job** operates at the database level and runs independently of any application. If the application server is down, the Job still executes on schedule.

Summary:

Reason	Object	Benefit
Consistent business logic across all systems	Stored Procedure	One version of the truth
Security enforced at the data source	View	Cannot be bypassed by any tool
Automatic audit regardless of change source	Trigger	100% coverage, no exceptions
Automation independent of applications	Scheduler Job	Runs even without user intervention

Conclusion:
Placing logic inside the database ensures that security, consistency, auditing, and automation are enforced at the source of the data — not at one specific application that could be bypassed, modified, or unavailable. This is why enterprise systems treat the database as the single point of control.

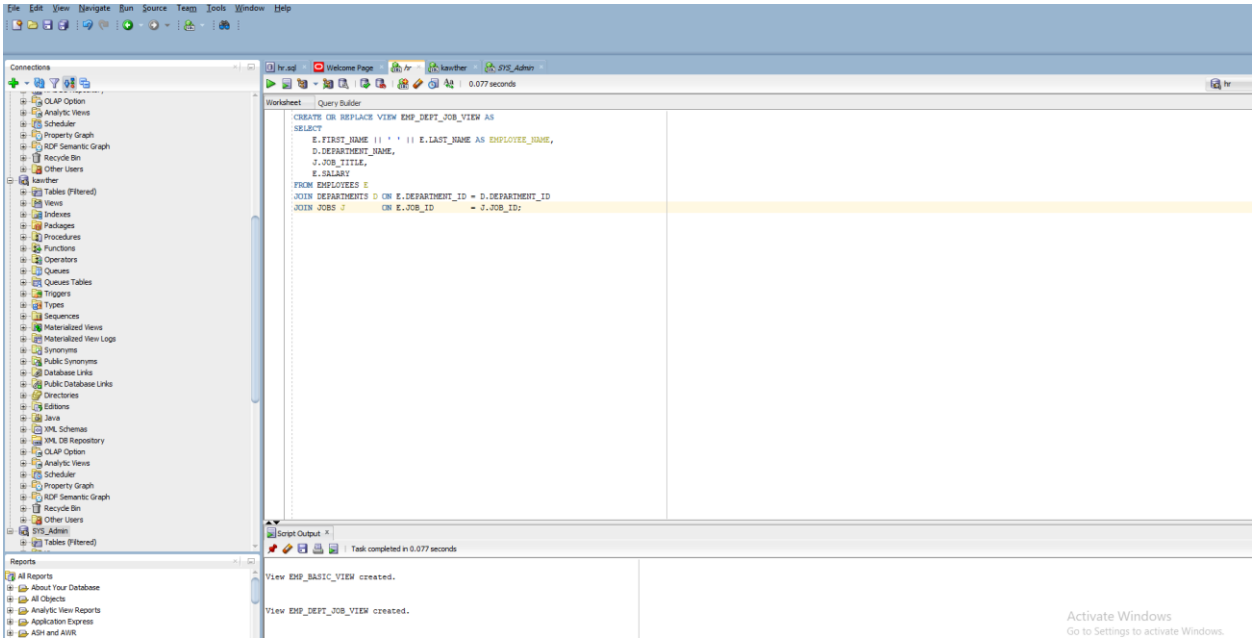
Part 5: View Implementation



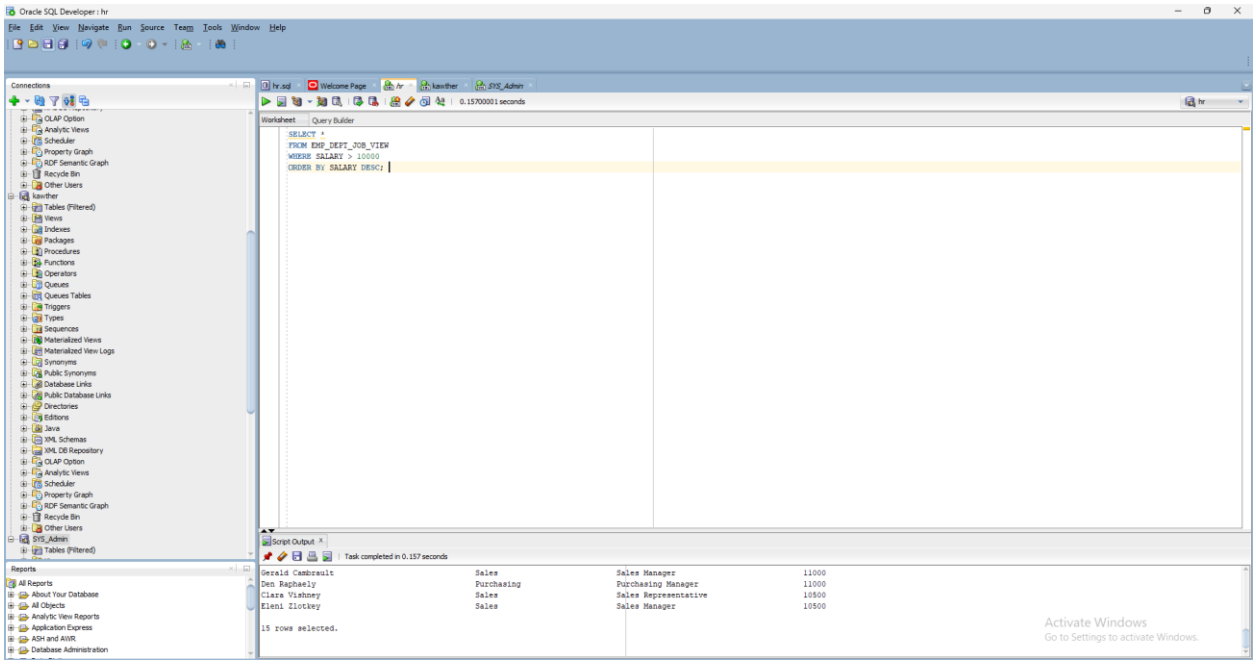
Task 1 – Create EMP_BASIC_VIEW:

The EMP_BASIC_VIEW was created successfully. The output displays Employee ID, Employee Name (first and last name concatenated), Department ID, and Salary for all 107 employees in the HR schema. The view hides all other columns in the EMPLOYEES table and is queried like a regular table.

Task 2 – Create Joined Employee View:


File Edit View Navigate Run Source Team Tools Window Help
Connections
hr.sql
Welcome Page
hr
kawther
SYS_Admin
0.077 seconds
Worksheet
Query Builder
CREATE OR REPLACE VIEW EMP_DEPT_JOB_VIEW AS
SELECT
E.FIRST_NAME || ' ' || E.LAST_NAME AS EMPLOYEE_NAME,
D.DEPARTMENT_NAME,
J.JOB_TITLE,
E.SALARY
FROM EMPLOYEES E
JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
JOIN JOBS J ON E.JOB_ID = J.JOB_ID;
Script Output
Task completed in 0.077 seconds
View EMP_BASIC_VIEW created.
View EMP_DEPT_JOB_VIEW created.
Reports
All Reports
About Your Database
All Objects
Analytic View Reports
Application Express
ASH and AWR
Activate Windows
Go to Settings to activate Windows.

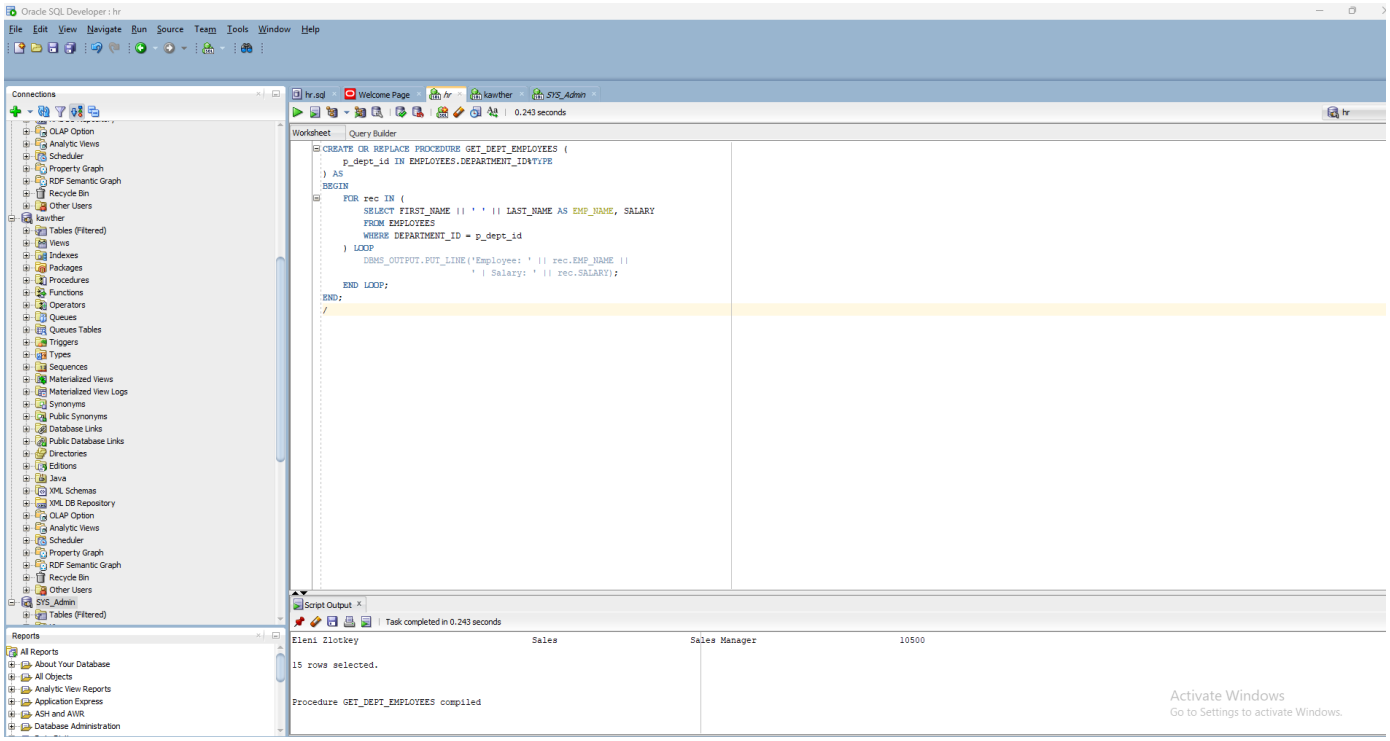
Task 3 – Display Employees Earning More Than 10000:

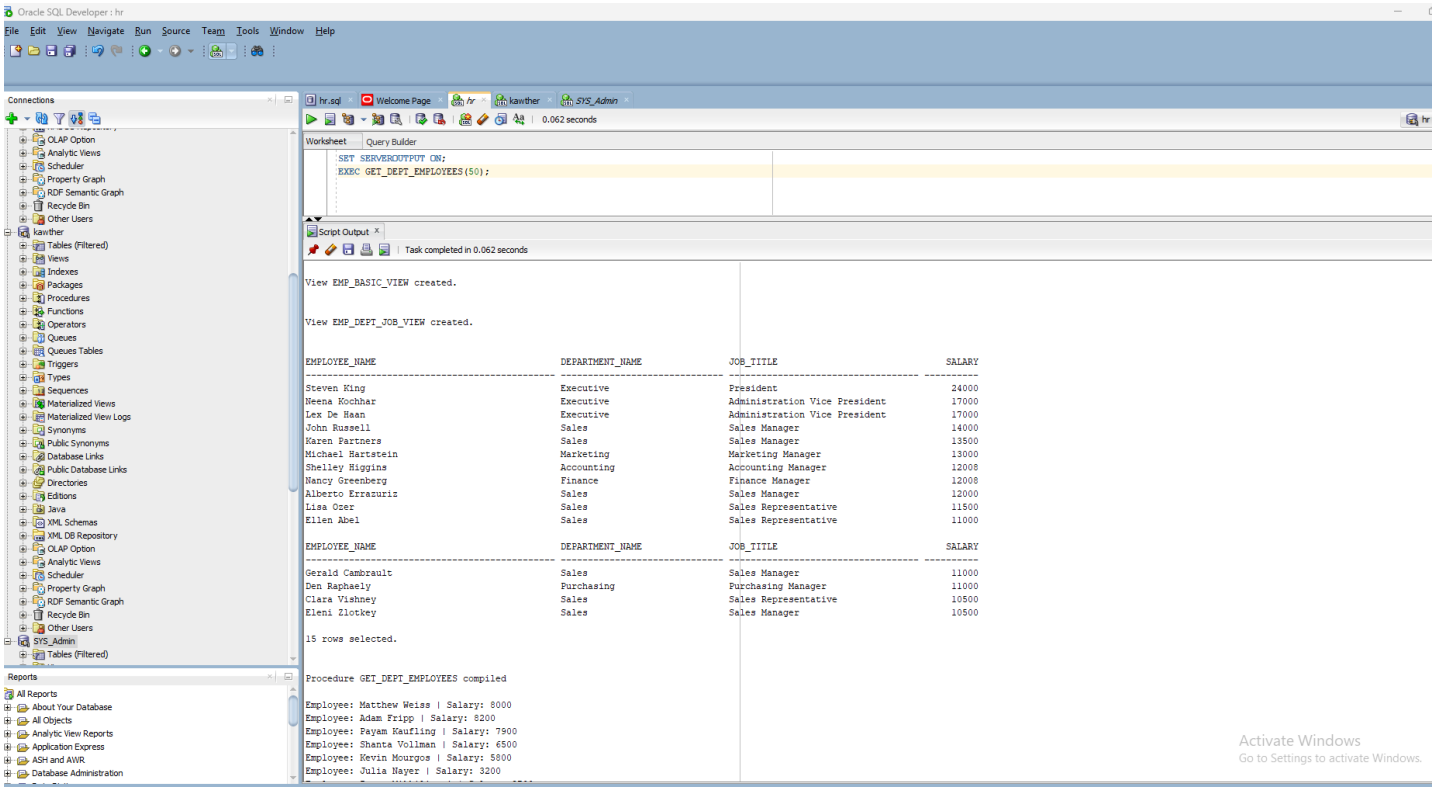


Task 4 – Why Use Views Instead of Querying EMPLOYEES Directly?

- Security: The View hides sensitive columns. Users cannot access salary or other restricted data beyond what the View exposes.
- Simplicity: The View already handles complex JOINs across EMPLOYEES, DEPARTMENTS, and JOBS. Management queries one object without needing to understand the schema.
- Consistency: All users see the same data format with the same column names and logic, reducing misinterpretation.
- Maintainability: If the table structure changes, only the View definition is updated — no changes needed in every user query or application.

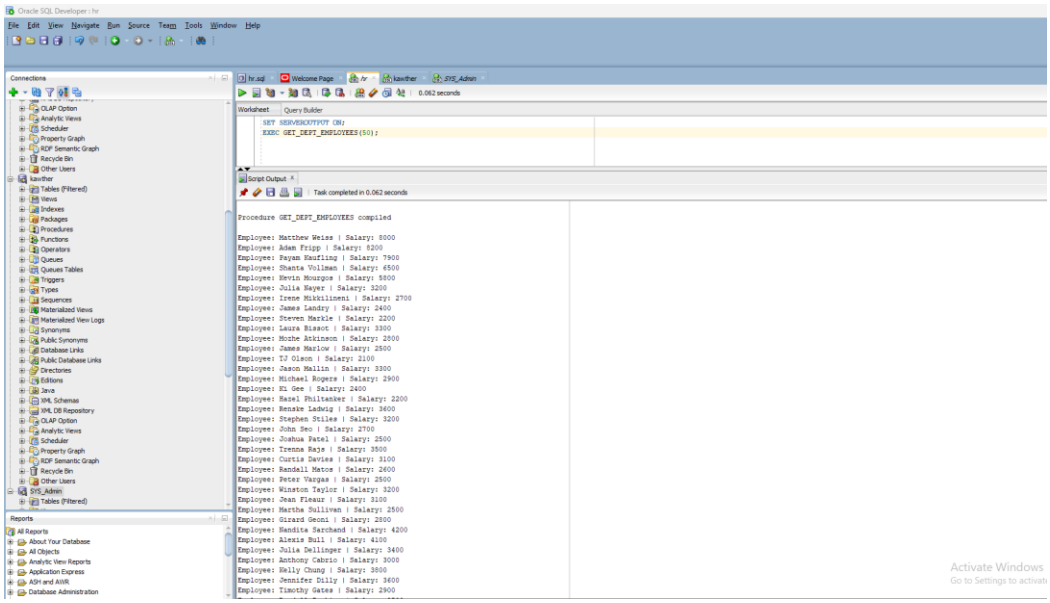
Part 6 – Procedure Implementation:

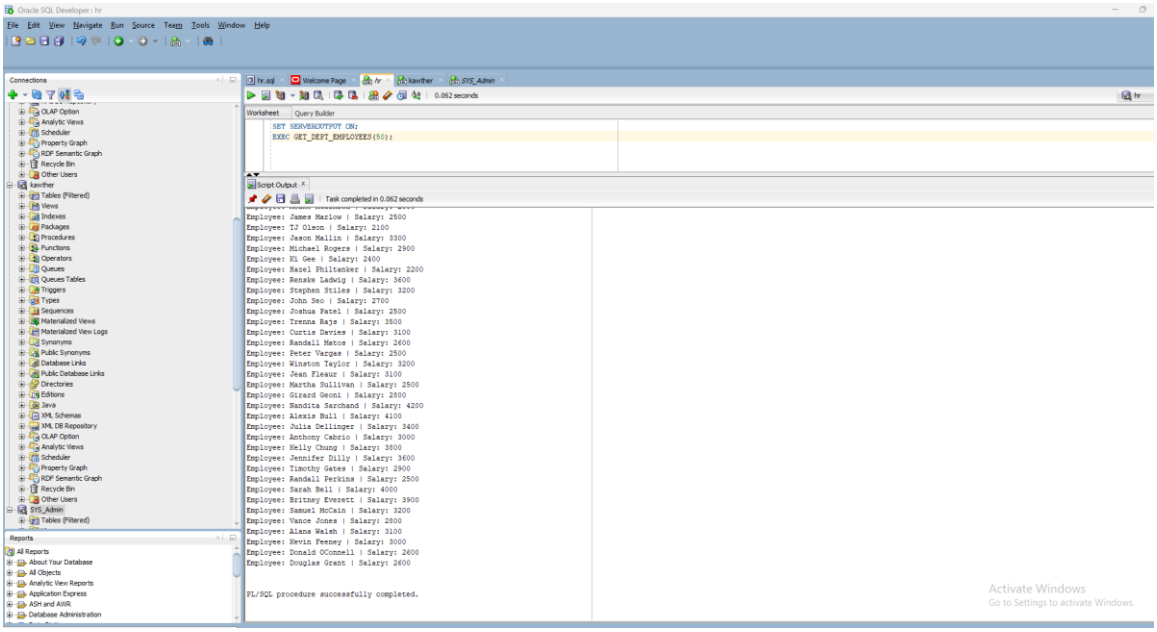




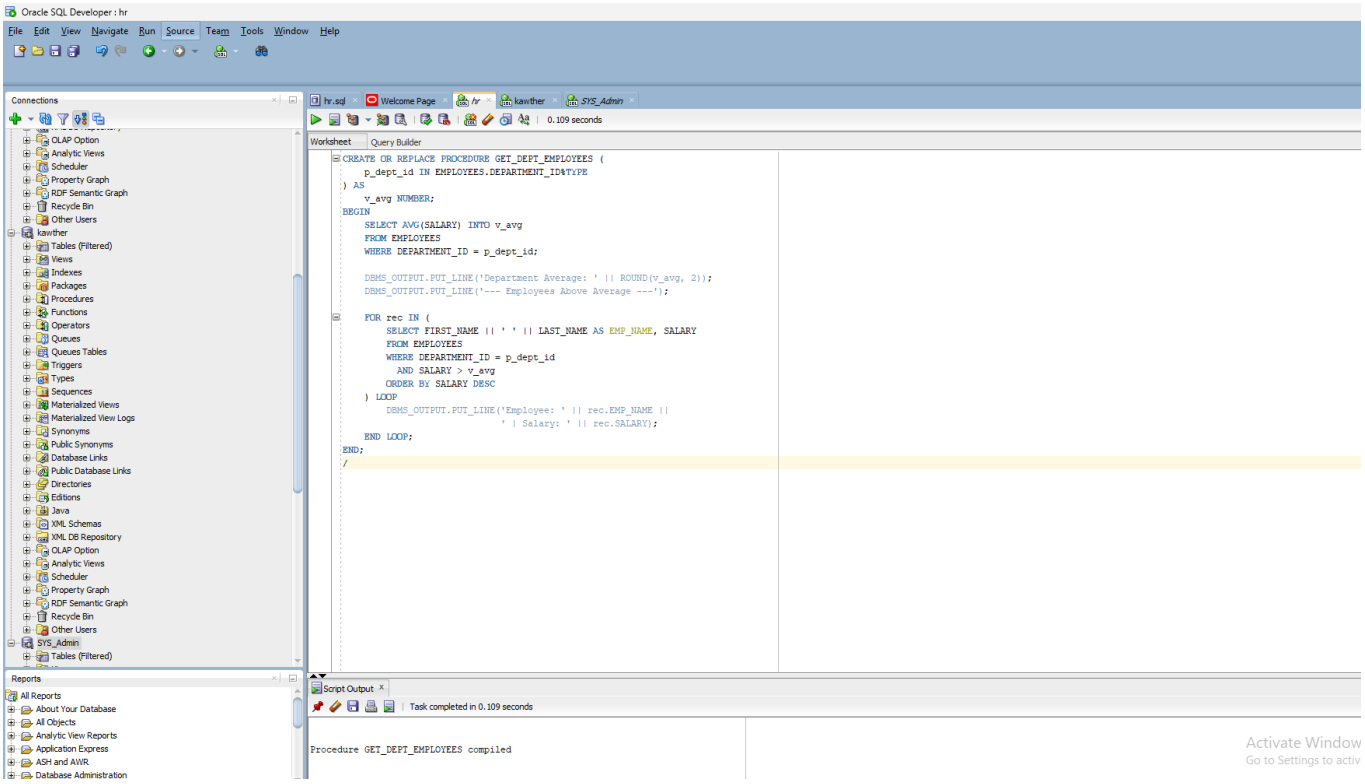
Task 5 – Create GET_DEPT_EMPLOYEES Procedure:

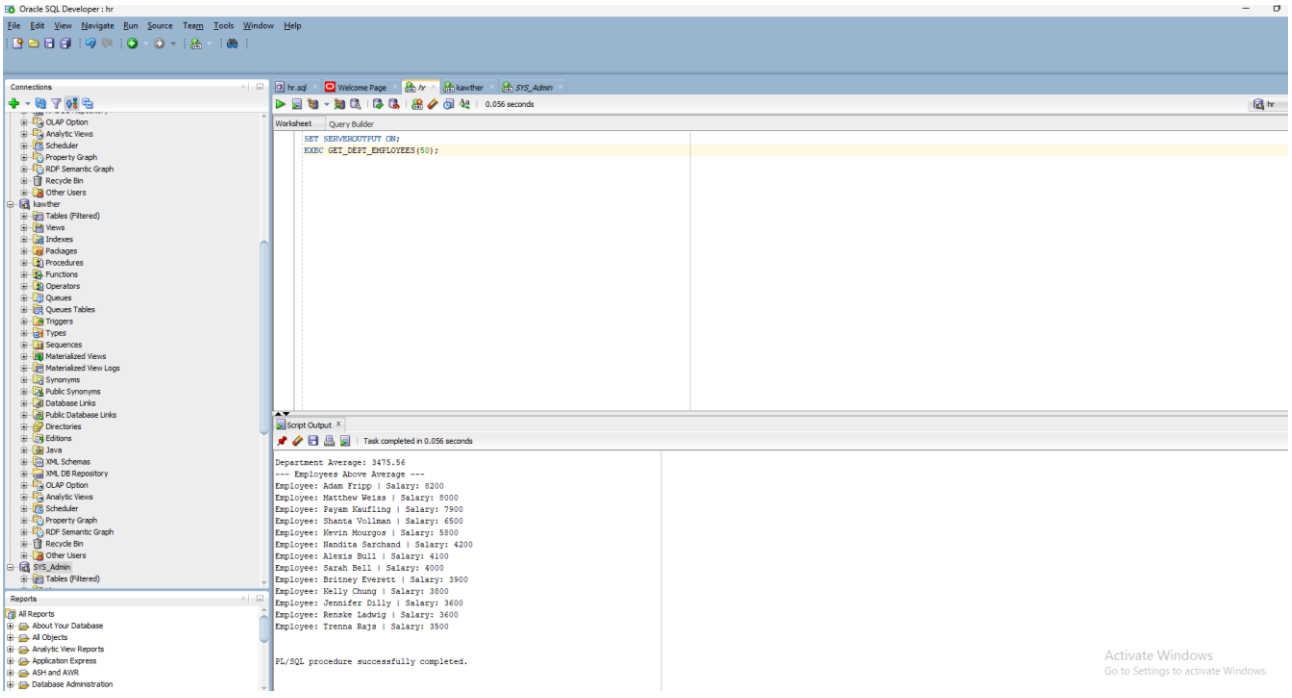
The GET_DEPT_EMPLOYEES Procedure was created and executed with Department ID 50. The Procedure accepts a department ID as input and returns the Employee Name and Salary for every employee in that department using DBMS_OUTPUT. The screenshot confirms the message “Procedure GET_DEPT_EMPLOYEES compiled” and shows the full list of Shipping department employees with their salaries.





Task 6 – Modify Procedure: Above Department Average Only :

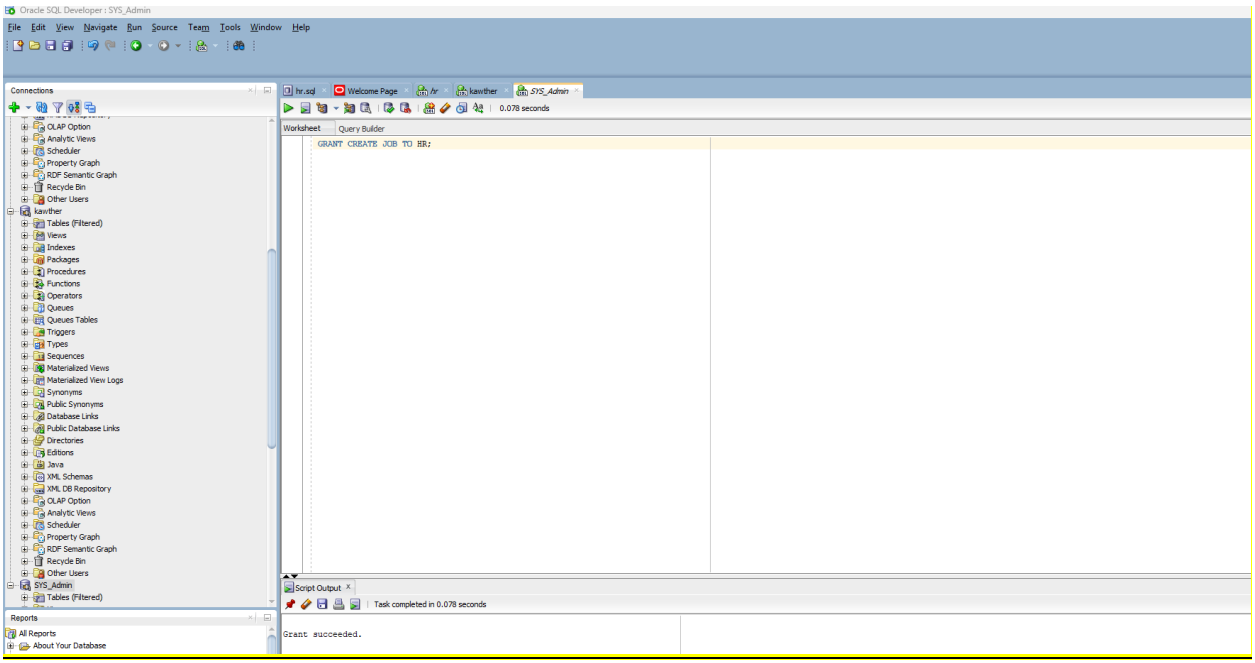




Task 7 – Why Is a Procedure Better Than Rewriting the Query?

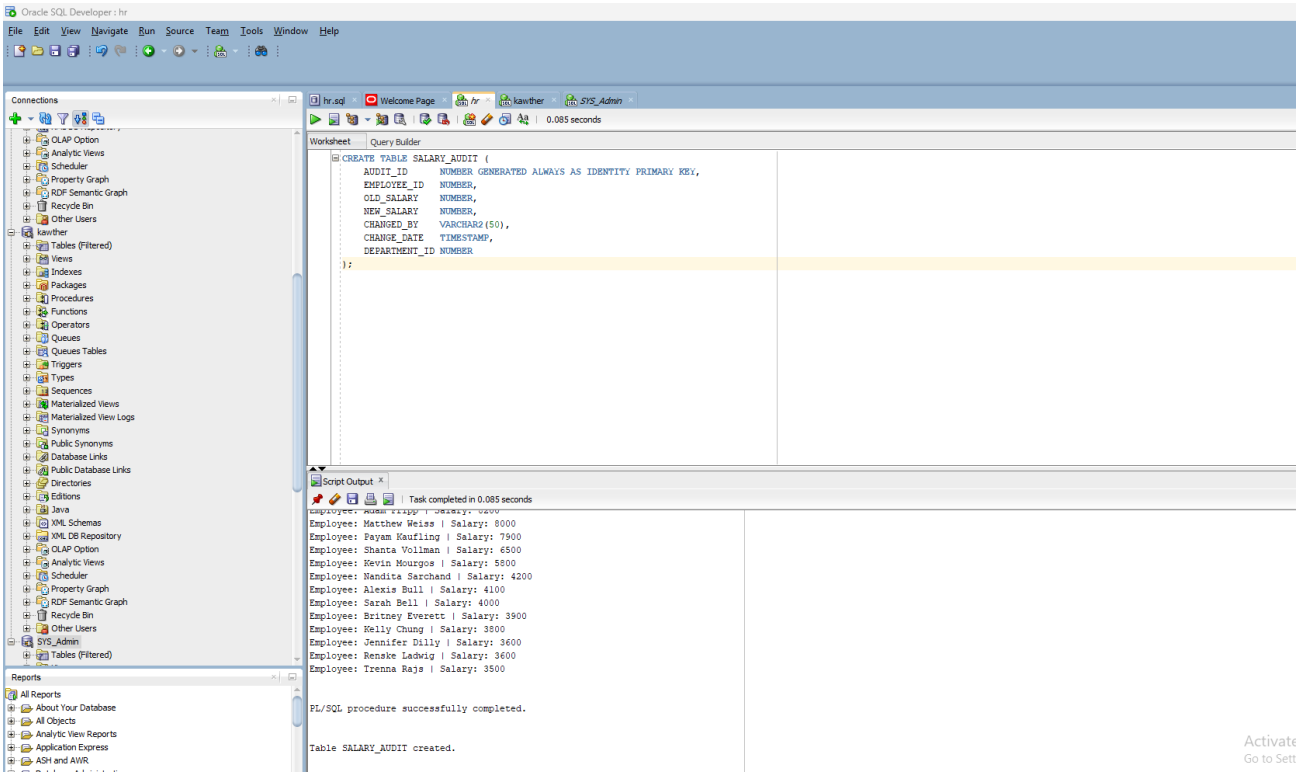
- Reusability: The Procedure is written once and called from any application, session, or scheduled job with different department IDs.
- Maintainability: If the business rule changes (e.g. filter by median instead of average), only the Procedure is updated — not every application.
- Security: Users are granted EXECUTE privilege only. They cannot see or modify the underlying SQL logic or access the tables directly.
- Performance: Oracle pre-compiles and caches the execution plan, reducing parsing overhead on repeated calls.

Part 7 – Trigger Implementation:



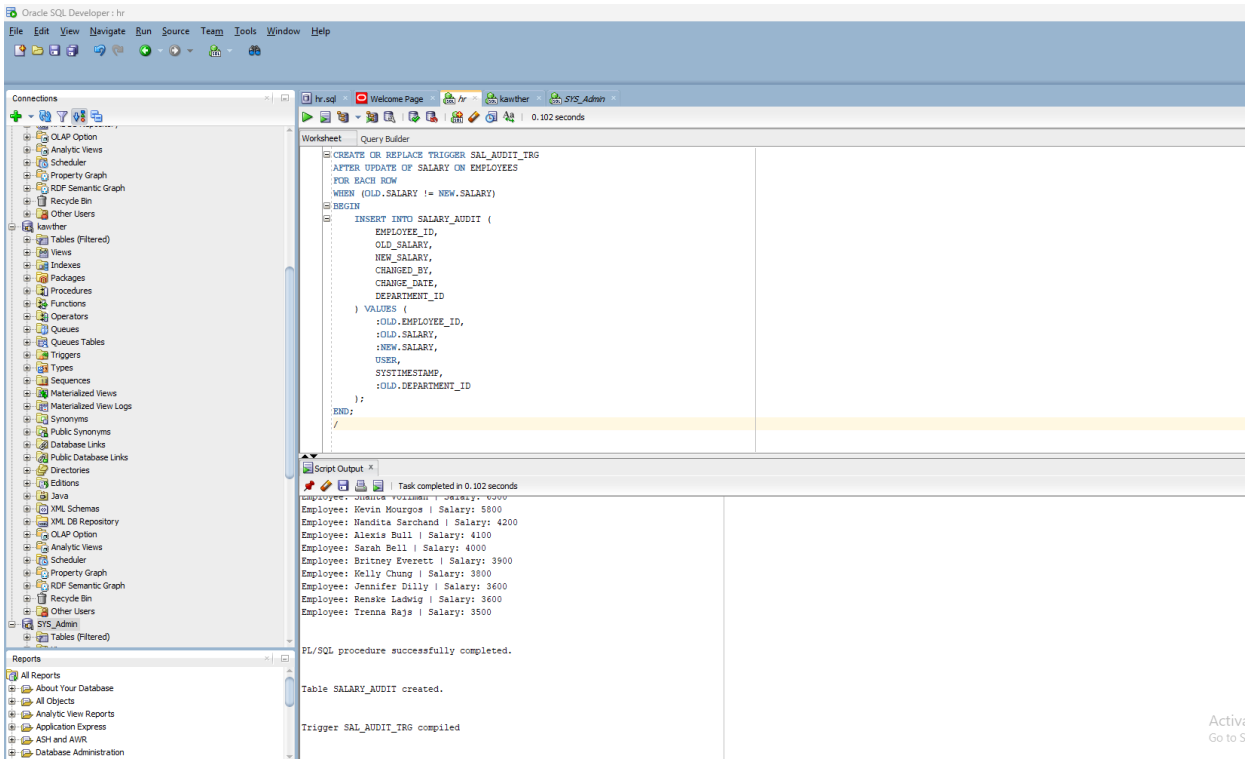
screenshot shows the GRANT CREATE JOB privilege granted to HR user, and confirms the SALARY_AUDIT table was created successfully in the HR schema to store Employee ID, Old Salary, New Salary, Changed By, Change Date, and Department ID.

Task 8 – Create SALARY_AUDIT Table:

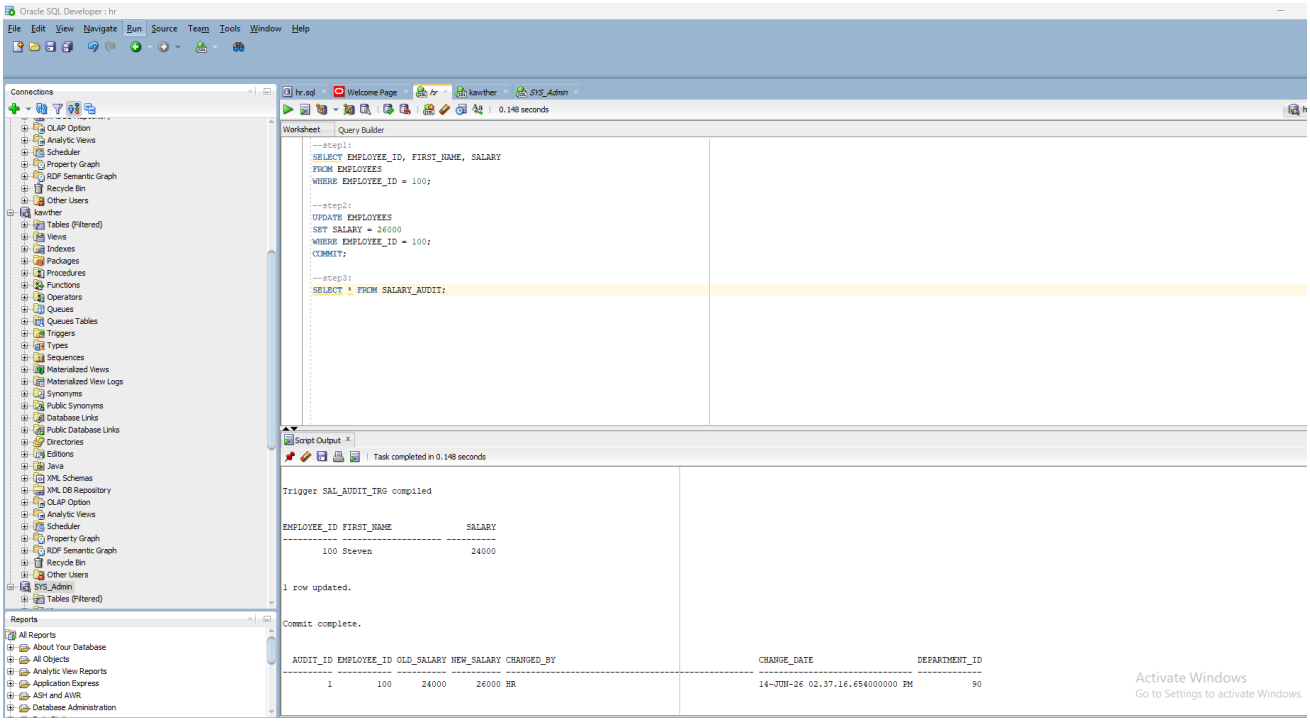


Task 9 – Create Salary Audit Trigger

Create a Trigger that automatically records every salary change including Username, Timestamp, and Department ID.



Task 10 – Test the Trigger:



Screenshot shows the SAL_AUDIT_TRG AFTER UPDATE trigger created on the EMPLOYEES table. Upon testing by updating Employee 100's salary from 24,000 to 26,000, the trigger automatically inserted an audit record. The SALARY_AUDIT table confirms the record with Old Salary: 24000, New Salary: 26000, Changed By: HR, and the exact timestamp.

Task 11 – Trigger Enhanced with Username and Timestamp:

The Trigger above already captures both Username and Timestamp using Oracle built-in variables:

- CHANGED_BY = USER — automatically captures the logged-in Oracle username at the time of the change.
- CHANGE_DATE = SYSTIMESTAMP — automatically captures the exact date and time with timezone precision.
- DEPARTMENT_ID = :OLD.DEPARTMENT_ID — captures the department the employee belonged to at the time of the salary change.

No additional modification is needed. The Trigger as written in Task 9 is the complete enhanced version covering Task 11 and Change Request 3.

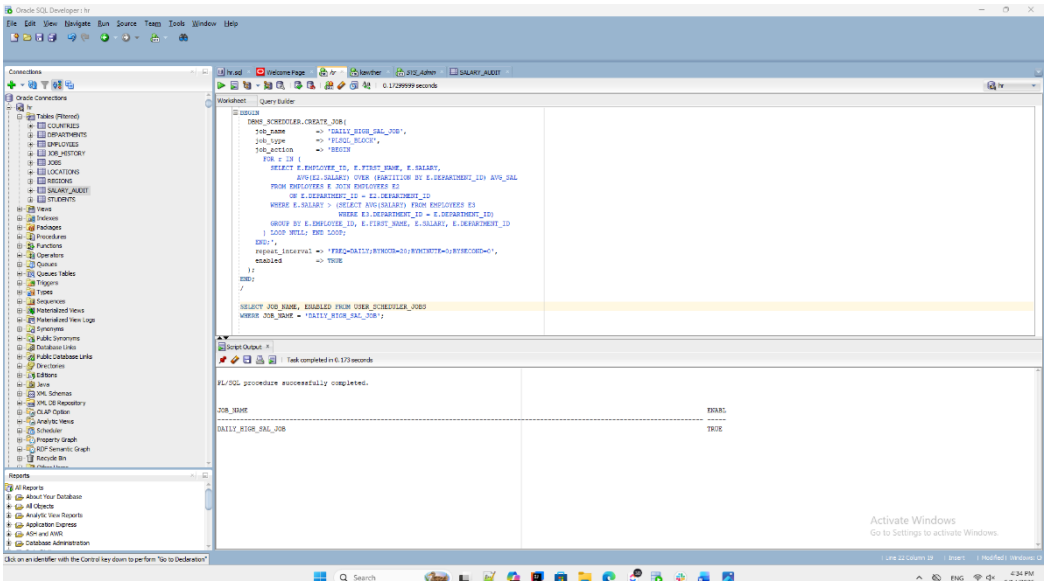
The trigger implementation shown above already captures Username (USER), Timestamp (SYSTIMESTAMP), and Department ID (:OLD.DEPARTMENT_ID) as confirmed in the SALARY_AUDIT record. No further modification was required.

Part 8 – Scheduler Job Design:

Note: Full Oracle Scheduler configuration is not required. The following designs provide Job Name, Schedule, and Logic for each business requirement.

Task 12 – Daily Salary Report (Every Day at 8:00 PM)

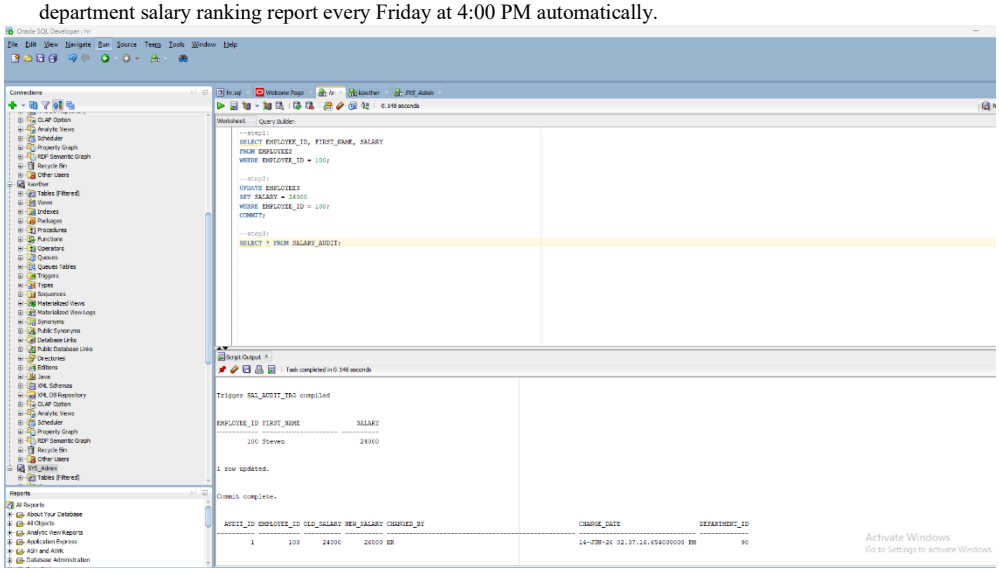
Property	Detail
Job Name	DAILY_HIGH_SAL_JOB
Schedule	Every day at 8:00 PM
Repeat Interval	FREQ=DAILY; BYHOUR=20; BYMINUTE=0; BYSECOND=0
Purpose	Identify employees whose salary exceeds their department average



Screenshot shows DAILY_HIGH_SAL_JOB created using DBMS_SCHEDULER with repeat interval FREQ=DAILY;BYHOUR=20;BYMINUTE=0;BYSECOND=0. The SELECT query from USER_SCHEDULER_JOBS confirms the job exists with ENABLED = TRUE.

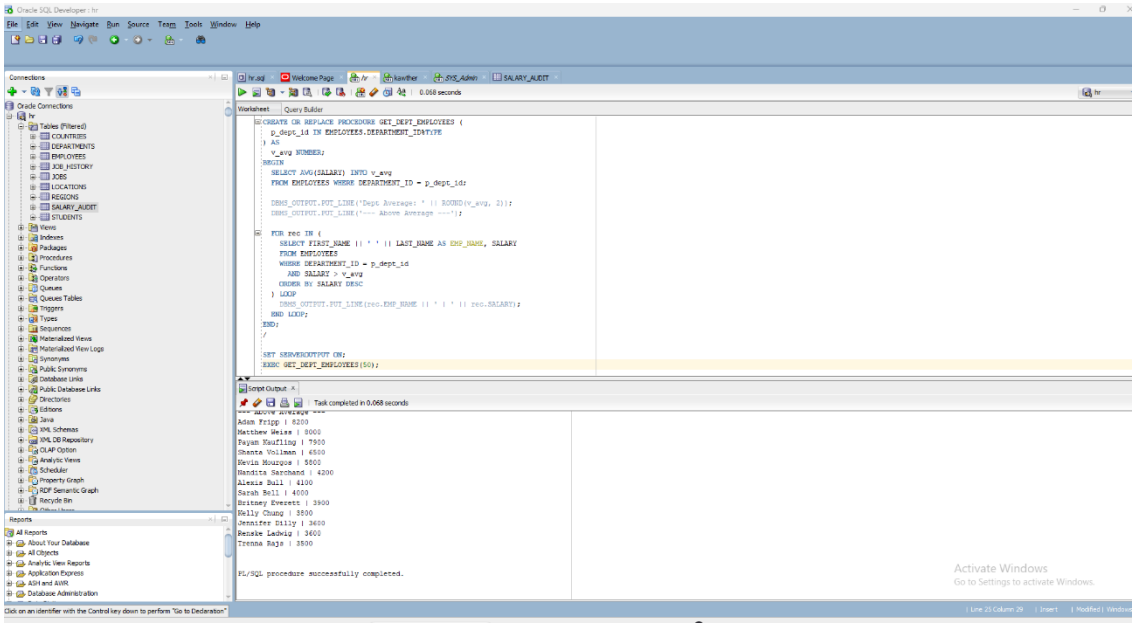
Task 13 – Weekly Department Salary Report (Every Friday at 4:00 PM):

Screenshot shows WEEKLY_DEPT_SAL_JOB created with repeat interval FREQ=WEEKLY;BYDAY=FRI;BYHOUR=16;BYMINUTE=0;BYSECOND=0. Confirmed as ENABLED = TRUE. This job generates a department salary ranking report every Friday at 4:00 PM automatically.



Task 14 – Monthly Stale Salary Report (First of Every Month):

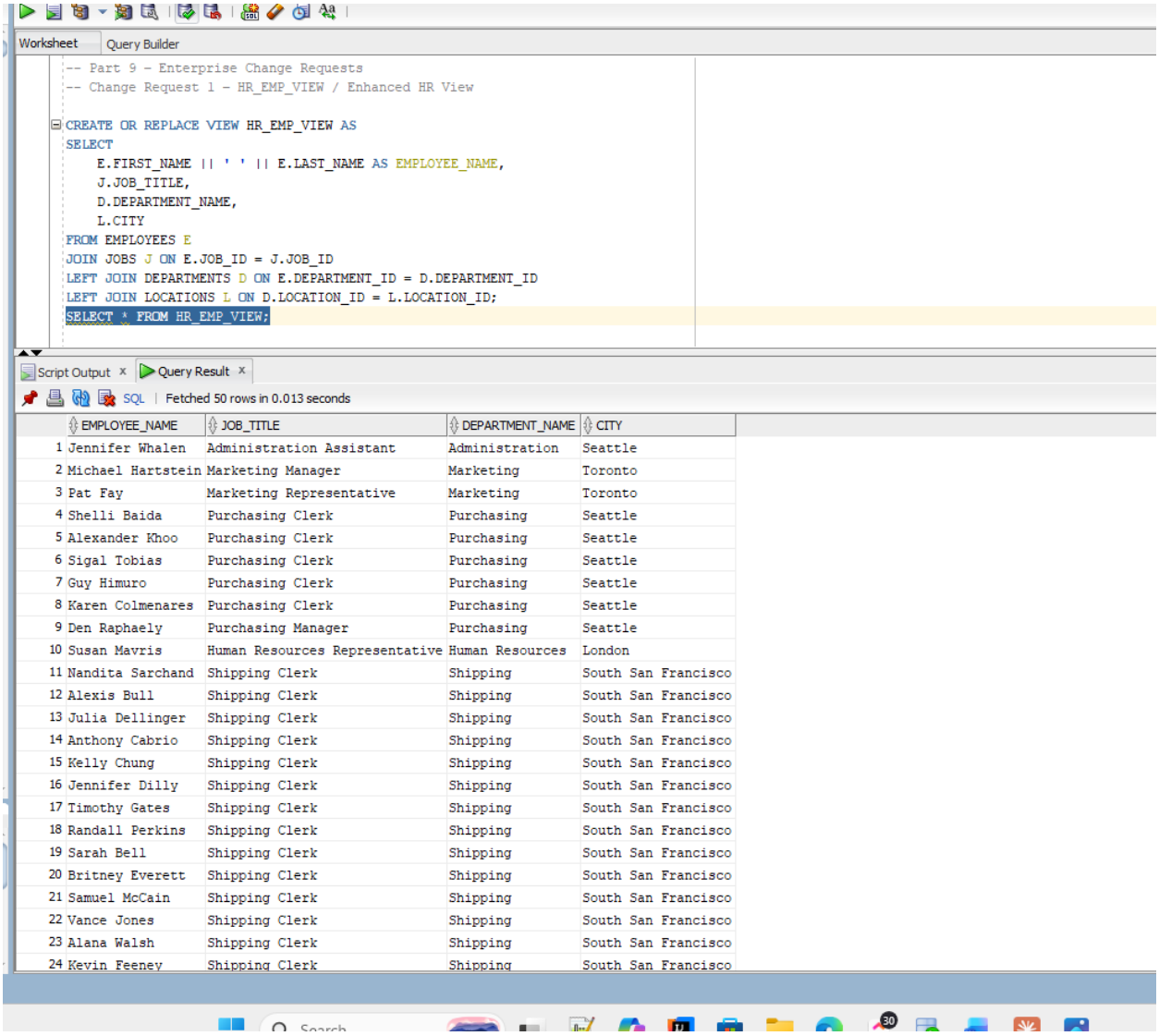
Screenshot shows MONTHLY_STALE_SAL_JOB created with repeat interval FREQ=MONTHLY;BYMONTHDAY=1;BYHOUR=6;BYMINUTE=0;BYSECOND=0. Confirmed as ENABLED = TRUE. This job identifies employees who have not received salary updates in the last six months, running automatically on the first day of every month.



Part 9 – Enterprise Change Requests:

Change Request 1 – HR_EMP_VIEW:

Screenshots show **HR_EMP_VIEW** created using a four-table JOIN across EMPLOYEES, JOBS, DEPARTMENTS, and LOCATIONS. The View displays Employee Name, Job Title, Department Name, and City without exposing Salary or the underlying base tables. Management accesses the required information through the View without any direct table exposure.



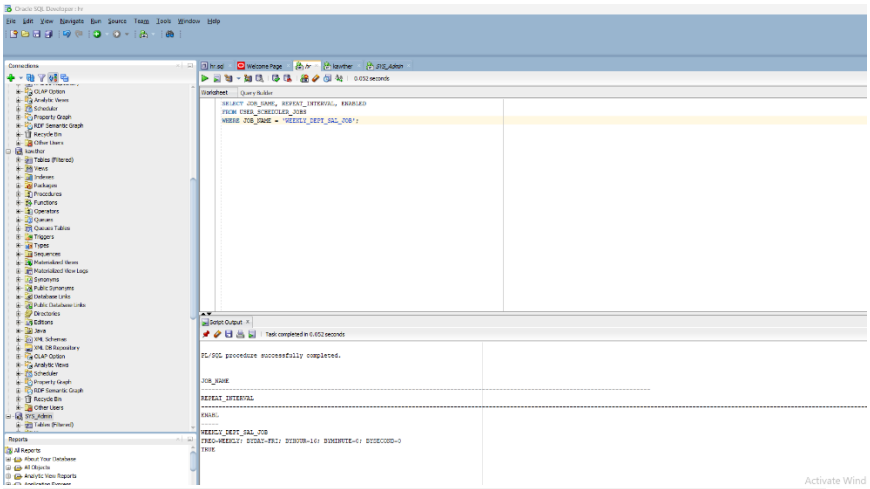
Worksheet | Query Builder

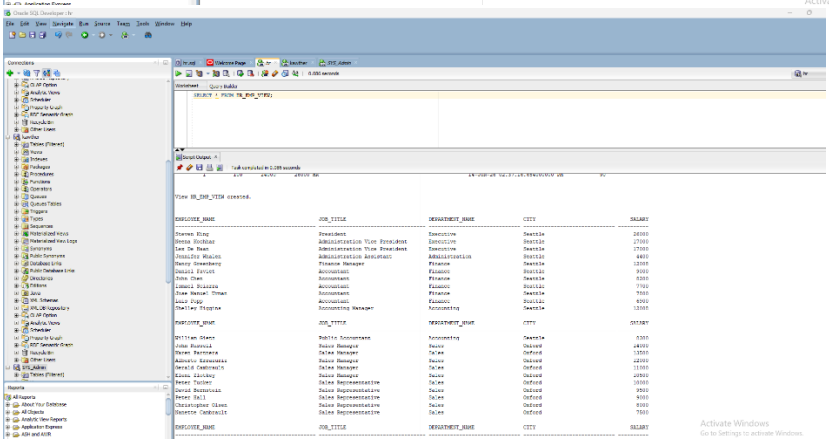
```
-- Part 9 – Enterprise Change Requests
-- Change Request 1 – HR_EMP_VIEW / Enhanced HR View

CREATE OR REPLACE VIEW HR_EMP_VIEW AS
SELECT
    E.FIRST_NAME || ' ' || E.LAST_NAME AS EMPLOYEE_NAME,
    J.JOB_TITLE,
    D.DEPARTMENT_NAME,
    L.CITY
FROM EMPLOYEES E
JOIN JOBS J ON E.JOB_ID = J.JOB_ID
LEFT JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
LEFT JOIN LOCATIONS L ON D.LOCATION_ID = L.LOCATION_ID;
SELECT * FROM HR_EMP_VIEW;
```

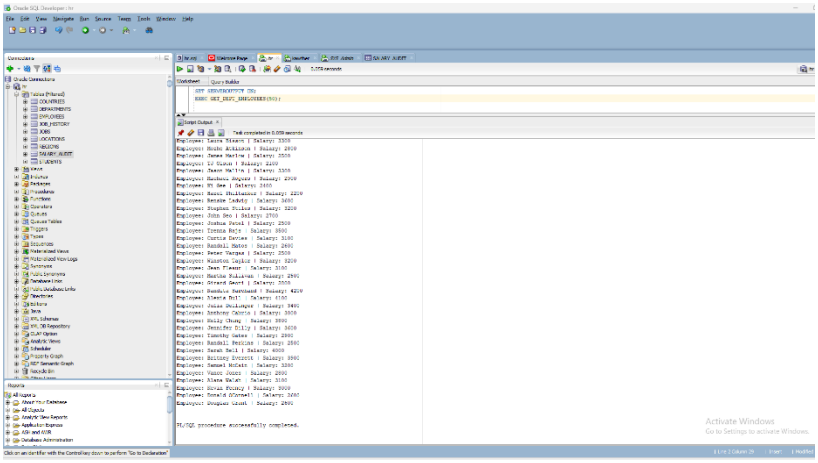
Script Output | Query Result | SQL | Fetched 50 rows in 0.013 seconds

	EMPLOYEE_NAME	JOB_TITLE	DEPARTMENT_NAME	CITY
1	Jennifer Whalen	Administration Assistant	Administration	Seattle
2	Michael Hartstein	Marketing Manager	Marketing	Toronto
3	Pat Fay	Marketing Representative	Marketing	Toronto
4	Shelli Baida	Purchasing Clerk	Purchasing	Seattle
5	Alexander Khoo	Purchasing Clerk	Purchasing	Seattle
6	Sigal Tobias	Purchasing Clerk	Purchasing	Seattle
7	Guy Himuro	Purchasing Clerk	Purchasing	Seattle
8	Karen Colmenares	Purchasing Clerk	Purchasing	Seattle
9	Den Raphaely	Purchasing Manager	Purchasing	Seattle
10	Susan Mavris	Human Resources Representative	Human Resources	London
11	Nandita Sarchand	Shipping Clerk	Shipping	South San Francisco
12	Alexis Bull	Shipping Clerk	Shipping	South San Francisco
13	Julia Dellinger	Shipping Clerk	Shipping	South San Francisco
14	Anthony Cabrio	Shipping Clerk	Shipping	South San Francisco
15	Kelly Chung	Shipping Clerk	Shipping	South San Francisco
16	Jennifer Dilly	Shipping Clerk	Shipping	South San Francisco
17	Timothy Gates	Shipping Clerk	Shipping	South San Francisco
18	Randall Perkins	Shipping Clerk	Shipping	South San Francisco
19	Sarah Bell	Shipping Clerk	Shipping	South San Francisco
20	Britney Everett	Shipping Clerk	Shipping	South San Francisco
21	Samuel McCain	Shipping Clerk	Shipping	South San Francisco
22	Vance Jones	Shipping Clerk	Shipping	South San Francisco
23	Alana Walsh	Shipping Clerk	Shipping	South San Francisco
24	Kevin Feeney	Shipping Clerk	Shipping	South San Francisco



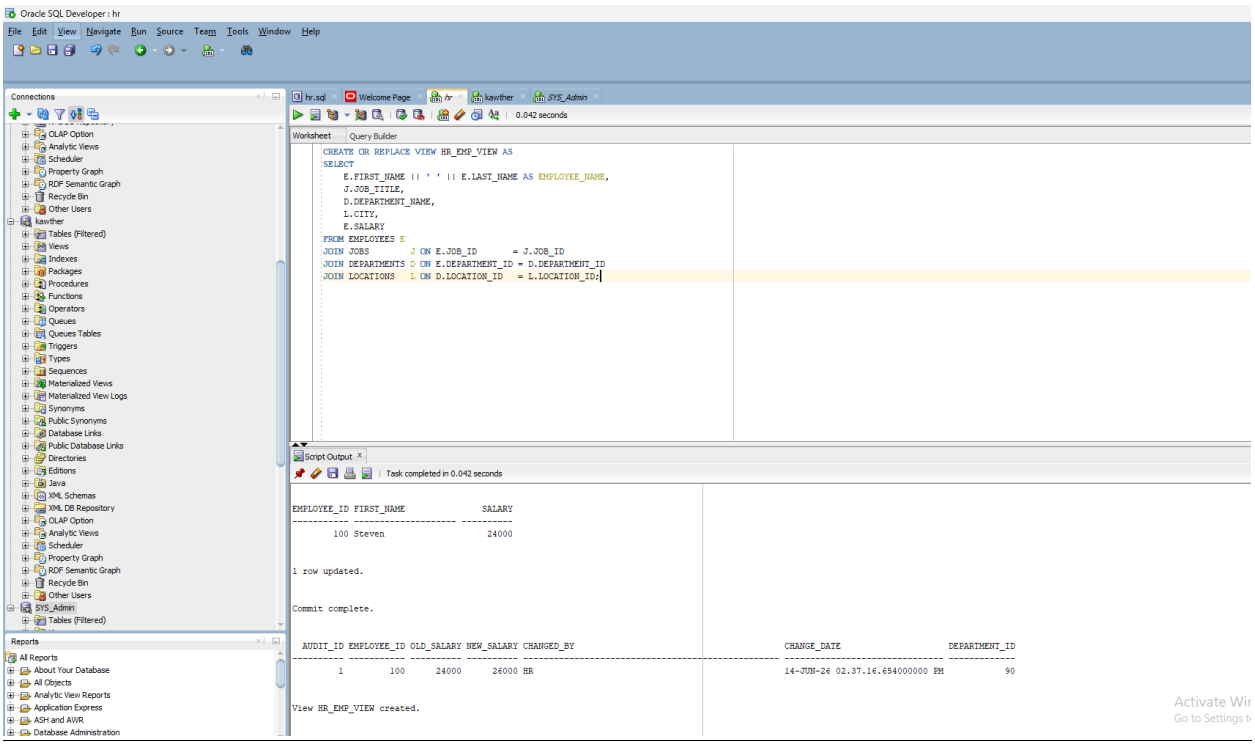


Change Request 2 – Procedure Modified:



Change Request 2 is fulfilled by the modified GET_DEPT_EMPLOYEES Procedure shown in Task 6. The Procedure already returns only employees earning above the department average, ordered by highest salary descending.

Change Request 3 – Trigger Enhanced:



Change Request 3 is already implemented. The SALARY_AUDIT record confirms that Username (HR), Timestamp (14-JUN-26), and Department ID (90) are all captured automatically by the trigger with every salary update.

Change Request 4 – Job Changed to Weekly:

The Scheduler Job was modified to run every Friday at 4:00 PM instead of every day. The WEEKLY_DEPT_SAL_JOB was created with repeat interval FREQ=WEEKLY;BYDAY=FRI;BYHOUR=16;BYMINUTE=0;BYSECOND=0 and confirmed as ENABLED = TRUE. The screenshot from Task 13 (Part 8) already demonstrates this configuration. The change from daily to weekly execution was applied by setting FREQ=WEEKLY and specifying BYDAY=FRI to target Friday only.

Part 10 – Final Enterprise Challenge

The following section presents a complete HR Auditing and Reporting Solution that addresses all five enterprise requirements using Views, Procedures, Triggers, and Scheduler Jobs.

Requirement 1 – HR users must not access tables directly.

Object Used: View (HR_EMP_VIEW). HR users are granted access to the View only. The View exposes Employee Name, Job Title, Department Name, and City without revealing Salary or the EMPLOYEES, DEPARTMENTS, JOBS, or LOCATIONS tables. No direct table access is permitted to any HR user.

Requirement 2 – Salary changes must be audited automatically.

Object Used: Trigger (SAL_AUDIT_TRG). An AFTER UPDATE Trigger on the EMPLOYEES table fires automatically every time a salary is changed. It inserts a record into the SALARY_AUDIT table capturing Employee ID, Old Salary, New Salary, Username (USER), Timestamp (SYSTIMESTAMP), and Department ID. This requires no manual action and works regardless of which application or user made the change.

Requirement 3 – Weekly reports must be generated automatically.

Object Used: Scheduler Job (WEEKLY_DEPT_SAL_JOB). This job runs every Friday at 4:00 PM using FREQ=WEEKLY;BYDAY=FRI;BYHOUR=16. It generates a department salary ranking report automatically with no user intervention required. Even if no user is logged in, the job executes on schedule at the database level.

Requirement 4 – Managers must access employee information through controlled access.

Object Used: Procedure (GET_DEPT_EMPLOYEES) and View (HR_EMP_VIEW). Managers call the Procedure with a department ID to retrieve employee data with salary. For dashboard access, managers query the HR_EMP_VIEW View which joins EMPLOYEES, DEPARTMENTS, JOBS, and LOCATIONS and returns all required fields. Neither access method exposes the underlying base tables directly.

Requirement 5 – Historical salary changes must be stored.

Object Used: Trigger + SALARY_AUDIT Table. Every salary update is permanently recorded in the SALARY_AUDIT table by the SAL_AUDIT_TRG trigger. Each record stores Employee ID, Old Salary, New Salary, Changed By (username), Change Date (timestamp with timezone), and Department ID. The full history is retained for compliance and audit review at any time.

Solution Summary:

Views: HR_EMP_VIEW displays employee name, job title, department name, and city without exposing salary. No user accesses base tables directly.

Procedures: GET_DEPT_EMPLOYEES allows managers to retrieve department employee data with salary through a single reusable call, ordered by highest salary descending.

Triggers: SAL_AUDIT_TRG automatically records every salary change into SALARY_AUDIT with full details including username, timestamp, and department ID, guaranteeing 100% audit coverage.

Scheduler Jobs: WEEKLY_DEPT_SAL_JOB runs every Friday at 4:00 PM to generate department salary reports. DAILY_HIGH_SAL_JOB runs every day at 8:00 PM to identify employees earning above their department average. MONTHLY_STALE_SAL_JOB runs on the 1st of every month to identify employees with no salary updates in six months. All jobs operate independently of any application or user session.

Conclusion: This solution places all security, auditing, and reporting logic inside the database, not in any single application. Views control access, the Trigger guarantees audit coverage, the Procedure provides reusable controlled access, and the Scheduler Jobs ensure automation independent of user availability. Together they form a complete enterprise-grade HR Auditing and Reporting System.